Assignment 1: Branch Predictor

COSC 6385 Advanced Computer Architecture
Pradham Mummaleti

Result: my Pentium M style hybrid predictor averages **7.861 MPKI** on the 20 traces. That is about 23% fewer mispredictions than using the bimodal table by itself (10.267 MPKI), and it matches all 22 steps of the Appendix B example.

1. The idea

The predictor has two parts that answer two different questions. The local (bimodal) table asks what this branch usually does. The global table asks what this branch does given what the last few branches did. The global table is more specific, so when it has an entry for the current situation I use it, and when it does not I fall back to the local table. This is the part of the Pentium M predictor the assignment asked for; I left out the loop predictor, the PIR hash and target prediction, and I used 2 ways instead of 4.

2. How one branch gets predicted

The easiest way for me to explain it is to walk through b1 from the Appendix B example, using the small version of the tables (4-bit local index, 2 row bits, 2 tag bits).

Step 1, get the address bits. b1 is at 0x44. Each hex digit is 4 bits, so 0x44 = 0100 0100. The example uses bits 5 to 2:

```
bit:  7 6 5 4 3 2 1 0
      0 1 0 0 0 1 0 0
          -------
          0 0 0 1   -> local index 1
```

Step 2, local prediction. Local counter 1 holds a value from 0 to 3. 0 or 1 means predict not taken, 2 or 3 means taken.

Step 3, global lookup. The global history register (GHR) holds the last 4 outcomes as bits, newest on the right. I XOR the address bits with it, then the top 2 bits are the row and the bottom 2 bits are the tag. Later in the trace, when the GHR is 1001:

```
address: 0 0 0 1
GHR:     1 0 0 1
XOR:     1 0 0 0   -> row 10 (= 2), tag 00
```

Step 4, hit or miss. Going to a row is not a hit by itself. One of the 2 ways in that row has to be valid and have the same tag, the same way a 2-way cache works. If it hits I use that entry's counter, otherwise I use the local prediction.

Step 5, update. The local counter always moves toward the real outcome (+1 if taken, -1 if not, staying between 0 and 3). If the global table hit, that counter is updated too. If it missed, the least recently used way in the row is taken over: the tag is replaced but the old counter value is kept, then updated. Last, the outcome is shifted into the GHR, for example 0110 followed by a not-taken branch becomes 1100.

For the real traces the tables are bigger: 4096 local counters, and 1024 rows x 2 ways with 6-bit tags and a 16-bit history, which is close to the real Pentium M sizes. I use the address from bit 0 instead of bit 2 because x86 instructions are not 4-byte aligned.

3. Checking it against Appendix B

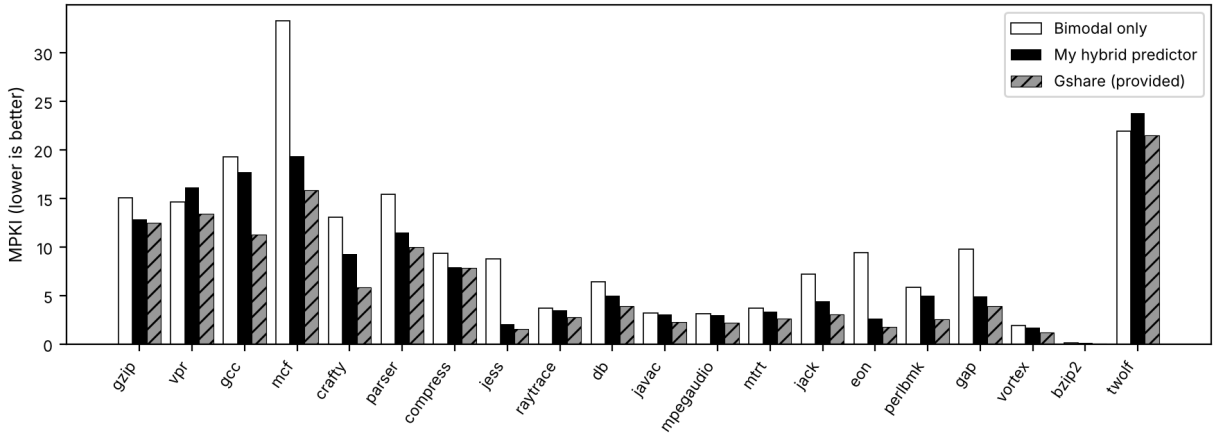
The table sizes are constructor parameters, so I can run the exact small example from the handout. src/appendixB_test.cpp runs the b1, b2, b3 sequence and compares every step to the handout's table. All 22 steps match. Here are the first 8:

Step	Branch	Actual	GHR	XOR	Row	Tag	Global	Final
1	b1	T	0000	0001	0	01	miss	NT
2	b3	T	0001	0101	1	01	miss	NT
3	b1	NT	0011	0010	0	10	miss	T
4	b2	NT	0110	1101	3	01	miss	T
5	b3	T	1100	1000	2	00	miss	NT
6	b1	T	1001	1000	2	00	hit	T
7	b3	T	0011	0111	1	11	miss	T
8	b1	NT	0111	0110	1	10	miss	T

Step 6 is the first global hit. b1 lands on the entry that b3 created in step 5, because their address and history XOR to the same row and tag. So the global table is really learning history patterns, not individual branches.

4. Results

I compared three predictors on the 20 traces: the bimodal table alone (same 4096 counters, no global part), my hybrid predictor, and the gshare sample that came with the infrastructure.



MPKI per trace for the three predictors

Trace	Bimodal only	My predictor	Gshare
164.gzip	15.031	12.833	12.473
175.vpr	14.673	16.147	13.415
176.gcc	19.313	17.695	11.254
181.mcf	33.271	19.359	15.837
186.crafty	13.097	9.300	5.837
197.parser	15.420	11.523	10.008
201.compress	9.368	7.945	7.831
202.jess	8.773	2.025	1.562
205.raytrace	3.704	3.480	2.756
209.db	6.437	4.956	3.909
213.javac	3.198	3.057	2.267
222.mpegaudio	3.166	3.014	2.188
227.mtrt	3.682	3.352	2.657
228.jack	7.175	4.403	3.033
252.eon	9.443	2.652	1.807
253.perlbmk	5.848	4.982	2.554
254.gap	9.760	4.926	3.926
255.vortex	1.930	1.682	1.222
256.bzip2	0.113	0.118	0.094
300.twolf	21.943	23.781	21.489
Average	**10.267**	**7.861**	**6.306**

5. What I noticed

Adding the global part helped the most on eon (9.44 to 2.65), jess (8.77 to 2.03) and mcf (33.27 to 19.36). These seem to be programs where branches depend on other recent branches, which is exactly what the GHR captures.

Gshare still wins on average (6.31), but it uses a 32K-entry table, about 8 KB, while mine is around 3.4 KB. On vpr and twolf my predictor is a little worse than the bimodal table alone. My guess is that with only 2 ways the global rows fill up and entries keep replacing each other, like conflict misses in a cache, so a new entry with an old counter value sometimes overrides a local prediction that was right. When I tried a few other table sizes (listed in the comments in my_predictor.h), adding global rows helped much more than growing the local table, which fits that explanation.

References

Assignment 1 handout and its Appendix B hybrid predictor example. Uzelac and Milenkovic, "Experiment Flows and Microbenchmarks for Reverse Engineering of Branch Predictor Structures," ISPASS 2009. The provided CBP2 infrastructure and gshare sample. No outside cache code was used.